

Mission Control Lesson 9

Public Deployment with Caddy and Optional Cloudflare — Complete Guide

This guide walks you through deploying your Mission Control to a public URL using Caddy as a reverse proxy. Follow the steps in order and you will skip all the troubleshooting from the video. Every command is ready to copy and paste. Throughout this guide, **mc.damianops.com** is used as the example domain and **72.62.134.173** as the example VPS IP address. Replace them with your own values wherever they appear — the next section explains how.

Before you start: This lesson is optional. If Tailscale (from the previous lesson) already gives you the access you need, you do not need to do any of this. Caddy is only for sharing Mission Control publicly with people who are not on your Tailscale network.

About the agent conversation: This guide follows the same approach that was demonstrated live in the lesson — first we discussed the deployment strategy with an AI agent, then we executed the setup manually. If you want to replicate the agent conversation yourself before doing the setup, see the **Appendix at the end of this guide** for the exact prompts that were used. Otherwise, just follow sections 1 through 14 for a clean step-by-step setup that skips all the troubleshooting.

Understanding the placeholders

Two things in this guide you will replace with your own values:

- **mc.damianops.com** — this is a subdomain. The domain that was bought is *damianops.com*, and **mc** is a subdomain that points to Mission Control. You create subdomains through your DNS settings (explained in section 2). You could use any subdomain you like — *app.yourdomain.com*, *control.yourdomain.com*, or whatever makes sense to you. You could also skip the subdomain and use the root domain directly, like *yourdomain.com*.
- **72.62.134.173** — this is the public IP address of the VPS. You can find yours in your hosting provider's control panel. On Hostinger, go to your VPS dashboard and look for **IPv4 address** on the overview page.

What's in this guide

- 1. Prerequisites
- 2. Buy a domain and point it to your VPS
- 3. Open the firewall port on your VPS
- 4. Connect to your VPS via SSH
- 5. Install Caddy on your VPS
- 6. Generate a self-signed certificate

- 7. Create the Caddyfile
- 8. Start Caddy and verify it's running
- 9. Make sure Mission Control is running
- 10. Test your public URL
- 11. Change your password any time
- 12. Optional: Use Cloudflare for extra polish and protection
- 13. Troubleshooting
- 14. Useful commands reference
- Appendix: Agent prompts used to design this setup

1. Prerequisites

Before you begin, make sure you have the following in place. You should have completed all previous Mission Control lessons, including setting up your VPS with Docker and the OpenClaw container running Mission Control on port 3007. You also need SSH access to your VPS, either through Tailscale or directly with root credentials. Lastly, you need a domain name. If you do not have one yet, the next section explains how to get one.

2. Buy a domain and point it to your VPS

You need a domain name that will point to your VPS. You can buy one from any registrar, such as Hostinger, Namecheap, GoDaddy, or Cloudflare Registrar. Domain names typically cost between 10 and 15 dollars per year.

Hostinger bonus: If you purchased a Hostinger VPS plan for 12 months or longer, you get one free domain for the first year included with your plan. Check your Hostinger account to claim it before buying a domain elsewhere.

Step 1: Buy your domain

Go to your registrar of choice and purchase a domain. This usually takes a few minutes.

Step 2: Create a DNS A record for your subdomain

Go to the DNS management section for your domain and add a new **A record**. This is where you create the **mc** subdomain that appears throughout this guide. Use the following values:

Field	Value
Type	A
Name / Host	mc (creates mc.yourdomain.com)
Value / Points to	Your VPS public IP address
TTL	3600 (or Automatic)

You can choose any name instead of **mc**. For example, if you use **app**, your URL will be *app.yourdomain.com*. If you want to skip the subdomain and use the root domain directly, use **@** in the Name field.

DNS propagation can take anywhere from a few minutes to a couple of hours. You can check if it has propagated by running this command in your local terminal (not the VPS):

```
ping mc.yourdomain.com
```

If it returns your VPS IP address, DNS is ready. If it says unknown host, wait a bit longer.

3. Open the firewall port on your VPS

The exact steps depend on your hosting provider, but the concept is the same everywhere: you need to allow incoming TCP traffic on **port 8444**. We use port 8444 because port 8443 is already in use by Docker for Mission Control.

Below are instructions for Hostinger, which is what this course uses. If you're on a different provider, look for the firewall or security groups section in your control panel and add the same rule.

- **DigitalOcean:** Networking → Firewalls → Add inbound rule for TCP port 8444
- **AWS EC2:** Security Groups → Edit inbound rules → Add TCP rule for port 8444
- **Linode:** Network → Firewalls → Add inbound rule for TCP port 8444
- **Vultr:** Firewall → Manage rules → Add TCP rule for port 8444
- **Self-hosted with UFW:** Run `sudo ufw allow 8444/tcp` on your server

Hostinger firewall setup

Step 1: Go to the Hostinger firewall panel

Log in to hpanel.hostinger.com, go to your VPS, and click on **Security** → **Firewall**.

Step 2: Edit your existing firewall or create a new one

If you already have a firewall configuration from previous lessons (like a Tailscale firewall), click **Edit** on it. Otherwise, click **Add firewall** and give it a name like *mission-control-firewall*.

Step 3: Add a rule for port 8444

Add the following firewall rule:

Field	Value
Action	Accept
Protocol	TCP
Port (or range)	8444
Source	Anywhere (0.0.0.0/0)

Step 4: Click Synchronize

After adding the rule, click the **Synchronize** button to apply the changes to your VPS. Wait around 30 seconds for the rule to take effect.

Important: Make sure your firewall also allows port 22 for SSH, and port 41641 for Tailscale if you use it. If you are creating a new firewall from scratch, add those rules too, otherwise you might lock yourself out of your VPS.

4. Connect to your VPS via SSH

All the commands from here onwards need to be run on your VPS, which means you need to connect to it via SSH. Open a terminal on your local computer (not on the VPS) and connect using one of the two methods below.

Option A: Connect via Tailscale (recommended)

If you set up Tailscale in the previous lesson, use your Tailscale IP. You can find it in the Tailscale admin console or by running *tailscale status* on your VPS.

```
ssh root@YOUR_TAILSCALE_IP
```

For example: *ssh root@100.67.190.10*

Option B: Connect via the VPS public IP

If you don't use Tailscale, connect directly to your VPS public IP address (the same IP you set in your DNS A record).

```
ssh root@YOUR_VPS_PUBLIC_IP
```

For example: *ssh root@72.62.134.173*

Stay connected for the rest of the guide. Once you are logged into your VPS, keep this SSH session open. Every command from sections 5 through 9 needs to be run on the VPS through this SSH connection. You'll know you're on the VPS when the prompt shows something like *root@srv1528049:~#*.

5. Install Caddy on your VPS

Paste the following commands into your SSH terminal to install Caddy and openssl:

```
sudo apt update
sudo apt install -y caddy openssl
```

This installs Caddy 2.x along with its dependencies. Once installed, Caddy is automatically enabled as a system service.

6. Generate a self-signed certificate

Instead of relying on Caddy's internal TLS generation (which has permission issues in some setups), we generate a self-signed certificate manually using openssl. This is more reliable and gives you full control over the certificate.

Paste this in your SSH terminal:

```
sudo mkdir -p /etc/caddy/certs
```

Then generate the certificate (valid for 1 year):

Before pasting, replace *mc.damianops.com* with your own domain in both the CN and subjectAltName fields:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
-keyout /etc/caddy/certs/cert.key \
-out /etc/caddy/certs/cert.crt \
-subj "/CN=mc.damianops.com" \
-addext "subjectAltName=DNS:mc.damianops.com,DNS:localhost"
```

Then set ownership and permissions:

```
sudo chown -R caddy:caddy /etc/caddy/certs
sudo chmod 600 /etc/caddy/certs/cert.key
sudo chmod 644 /etc/caddy/certs/cert.crt
```

7. Create the Caddyfile

The Caddyfile tells Caddy what to do. We want Caddy to listen on port 8444 with HTTPS (using your self-signed certificate), protect everything with basic auth, and forward requests to Mission Control on port 3007.

Step 1: Generate a random password and hash it (paste in SSH terminal):

```
PASSWORD="$(openssl rand -base64 24 | tr -d '\n')"
```

```
HASH="$(caddy hash-password --plaintext "$PASSWORD")"
```

```
echo "USERNAME: admin"
```

```
echo "PASSWORD: $PASSWORD"
```

Save this password right now! Copy the password that gets printed to the terminal and store it in a password manager. You will not be able to recover it later (only the hash is stored in the Caddyfile).

Step 2: Create the Caddyfile. Replace *mc.damianops.com* with your own domain before pasting:

```
sudo tee /etc/caddy/Caddyfile >/dev/null <<EOF
{
  auto_https off
}

localhost:8444, mc.damianops.com:8444 {
  tls /etc/caddy/certs/cert.crt /etc/caddy/certs/cert.key

  basicauth {
    admin $HASH
  }

  reverse_proxy 127.0.0.1:3007
}
EOF
```

The `$HASH` variable gets automatically substituted with the hash from step 1, which is why both commands need to run in the same SSH session.

8. Start Caddy and verify it's running

Enable and restart Caddy (paste in SSH terminal):

```
sudo systemctl enable caddy
sudo systemctl restart caddy
```

Check that Caddy is running properly:

```
sudo systemctl status caddy --no-pager
```

You should see **active (running)** in green. If you see **failed** in red, jump to the troubleshooting section at the end of this guide.

Test HTTPS locally on the server:

```
curl -k -I https://localhost:8444
```

You should see **HTTP/2 401** along with a **www-authenticate: Basic realm="restricted"** header. This confirms that Caddy is serving HTTPS properly and basic auth is protecting the site.

9. Make sure Mission Control is running

Caddy forwards requests to Mission Control on port 3007. If Mission Control is not running inside the container, Caddy will return a 502 Bad Gateway error. In the Task Board lesson earlier in this course, you set up PM2 (a process manager) to keep Mission Control running and auto-restart it if it crashes. This section makes sure PM2 is still managing Mission Control properly.

About PM2 and container restarts: PM2 keeps Mission Control running within the container, and will auto-restart the process if it crashes. However, if the whole Docker container gets destroyed and recreated (for example during a Hostinger deploy), you may need to re-run the commands in this section to bring Mission Control back up.

Step 1: Check if Mission Control is responding

Paste in your SSH terminal:

```
curl -I http://127.0.0.1:3007
```

If you get HTTP/1.1 200 OK, you're good — skip to section 10. **If you get Connection reset by peer or Connection refused**, Mission Control is not running and you need to follow the rest of this section.

Step 2: Find your container name

```
docker ps
```

Look for the container running the *hvps-openclaw* image. The name will look something like *openclaw-ij7b-openclaw-1*. **Use your actual container name in all commands below.**

Step 3: Check the PM2 process list

See if PM2 is already managing Mission Control (replace the container name with yours):

```
docker exec openclaw-ij7b-openclaw-1 sh -c \  
"cd /data/.openclaw/workspace/mission-control && \  
./node_modules/.bin/pm2 list"
```

You'll see one of three outcomes. Find yours below and follow the matching step:

Outcome A: mission-control is listed as 'online'

PM2 thinks Mission Control is running but the port isn't responding. Restart it:

```
docker exec openclaw-ij7b-openclaw-1 sh -c \  
"cd /data/.openclaw/workspace/mission-control && \  
./node_modules/.bin/pm2 restart mission-control"
```

Outcome B: mission-control is listed as 'stopped' or 'errored'

Start it again:

```
docker exec openclaw-ij7b-openclaw-1 sh -c \  
"cd /data/.openclaw/workspace/mission-control && \  
./node_modules/.bin/pm2 start mission-control"
```


Outcome C: mission-control is not in the list at all

This happens if the container was recreated and PM2 state was lost. Start it fresh:

```
docker exec openclaw-ij7b-openclaw-1 sh -c \  
"cd /data/.openclaw/workspace/mission-control && \  
PORT=3007 HOSTNAME=0.0.0.0 \  
./node_modules/.bin/pm2 start npm --name mission-control -- start"
```

Step 4: Save the PM2 state

After starting or restarting, save the PM2 process list so it can be restored later:

```
docker exec openclaw-ij7b-openclaw-1 sh -c \  
"cd /data/.openclaw/workspace/mission-control && \  
./node_modules/.bin/pm2 save"
```

Step 5: Wait a few seconds, then test again

```
sleep 10  
curl -I http://127.0.0.1:3007
```

You should now see **HTTP/1.1 200 OK** from Next.js. If you're still getting Connection reset, check the PM2 logs:

```
docker exec openclaw-ij7b-openclaw-1 sh -c \  
"cd /data/.openclaw/workspace/mission-control && \  
./node_modules/.bin/pm2 logs mission-control --lines 30 --nostream"
```

Fallback option: If PM2 is not installed in your container at all (for example if you skipped the Task Board lesson), you can start Mission Control temporarily with just npm. It won't auto-restart but it will get you running for testing:

```
docker exec -d openclaw-ij7b-openclaw-1 sh -c \  
"cd /data/.openclaw/workspace/mission-control && \  
PORT=3007 npm start > /tmp/mc.log 2>&1"
```

10. Test your public URL

Open a fresh incognito window in your browser (on your local computer, not the VPS) and go to:

```
https://mc.yourdomain.com:8444
```

You will see this flow:

- **1. Certificate warning** — The browser shows "Your connection is not private" with the error code `NET::ERR_CERT_AUTHORITY_INVALID`. This is expected because we used a self-signed certificate. Click **Advanced**, then **Proceed to mc.yourdomain.com**.
- **2. Login prompt** — A basic auth dialog appears asking for username and password. Enter **admin** as the username and the password you saved in section 7.

- **3. Mission Control loads** — You should now see your full Mission Control interface running on your public URL.

That's it! You can now access Mission Control from anywhere in the world. Phone, tablet, laptop, someone else's computer — as long as they have the URL and the password, they can log in.

11. Change your password any time

You can update the basic auth password whenever you want. This is useful if you want to set a password you can actually remember, or if you want to rotate it for security. The commands below only take a few seconds to run.

Before running any of the password commands below, you need to be connected to your VPS via SSH. If you're not currently connected, open a terminal on your local computer and run:

```
ssh root@YOUR_TAILSCALE_IP_OR_VPS_IP
```

Once you see the VPS prompt (like *root@srv1528049:~#*), you're ready to run the commands below.

Option A: Generate a new random password

Paste this in your SSH terminal:

```
PASSWORD="$(openssl rand -base64 24 | tr -d '\n')"  
HASH="$(caddy hash-password --plaintext "$PASSWORD")"  
sudo sed -i "s|admin .*|admin $HASH|" /etc/caddy/Caddyfile  
sudo systemctl reload caddy  
echo "USERNAME: admin"  
echo "PASSWORD: $PASSWORD"
```

Option B: Set your own password

Replace **YourNewPassword123** with whatever you want, then paste in your SSH terminal:

```
PASSWORD="YourNewPassword123"  
HASH="$(caddy hash-password --plaintext "$PASSWORD")"  
sudo sed -i "s|admin .*|admin $HASH|" /etc/caddy/Caddyfile  
sudo systemctl reload caddy  
echo "USERNAME: admin"  
echo "PASSWORD: $PASSWORD"
```

The new password takes effect immediately after the reload command. If your browser was already logged in with the old password, close the tab and open a fresh incognito window to log in with the new password.

Tip: Store your password in a password manager like 1Password or Bitwarden. If you lose it, you cannot recover it — you will need to generate a new one using the commands above.

12. Optional: Use Cloudflare for extra polish and protection

What you built in the previous sections is a complete, working setup. Mission Control is publicly accessible, protected by HTTPS and basic auth, and ready to use. This section is about adding an optional upgrade layer on top — Cloudflare. Everything you already set up stays exactly the same, Cloudflare just sits in front of it and makes the experience nicer for your visitors.

What Cloudflare gives you on top of Caddy

- **Cleaner URL** — no more `:8444` at the end. Just `https://mc.yourdomain.com`.
- **Trusted SSL certificate** — no more certificate warning when you visit the site. The browser sees a valid certificate from Cloudflare and loads the page normally.
- **Hides your VPS IP address** — attackers can't see or directly target your server because traffic goes through Cloudflare first.
- **DDoS protection** — Cloudflare absorbs and filters malicious traffic before it reaches your server.
- **CDN and caching** — static assets like images and CSS are cached on Cloudflare's global network, making the site load faster for visitors far from your VPS.
- **Analytics** — you can see how many visits your Mission Control is getting, where they come from, and what browsers they use.
- **Firewall rules** — block specific countries, IP addresses, or bad bots with a few clicks.
- **Cloudflare Access (extra)** — you can add Google, GitHub, or email-link login on top of your basic auth for even better security.

Your Caddy setup stays important. Caddy is still the server that actually runs Mission Control, handles basic auth, and forwards traffic to port 3007. Cloudflare just sits in front of Caddy like a bouncer at a club — it makes the entrance nicer and blocks troublemakers, but Caddy is still the one doing the real work inside.

Step 1: Create a free Cloudflare account

Go to cloudflare.com and sign up for a free account. The free tier includes everything we need — trusted SSL, DDoS protection, CDN, and analytics.

Step 2: Add your domain to Cloudflare

In the Cloudflare dashboard, click **Add a site**, enter your domain (e.g. `yourdomain.com`), and follow the wizard. Cloudflare scans your existing DNS records and shows them to you.

Step 3: Update your nameservers

Cloudflare gives you two nameservers (something like `alex.ns.cloudflare.com` and `kate.ns.cloudflare.com`). Go to your domain registrar and replace the existing nameservers with these two. This can take up to 24 hours to propagate, but is usually done within an hour.

Step 4: Configure the DNS record in Cloudflare

In Cloudflare's DNS settings, make sure your A record for **mc** points to your VPS IP address (the same IP you used before). Click the cloud icon next to the record to make it **orange (proxied)** instead of grey (DNS only). This tells Cloudflare to proxy the traffic through its network.

Step 5: Set SSL/TLS mode to Full

Go to **SSL/TLS** → **Overview** and set the encryption mode to **Full**. This tells Cloudflare to use HTTPS both ways — between the visitor and Cloudflare, and between Cloudflare and your Caddy server. Your Caddy is already set up to serve HTTPS on port 8444, which is exactly what Cloudflare needs.

Step 6: Configure the connection to your Caddy backend

Cloudflare accepts visitor traffic on port 443 (standard HTTPS) but your Caddy is listening on 8444. You have two ways to connect them:

- **Option 1 (easier):** Use a Cloudflare Worker or Origin Rule to rewrite the request port from 443 to 8444 when forwarding to your server. This keeps your Caddy setup unchanged.
- **Option 2 (simpler server side):** Update your Caddyfile to listen on a port that Cloudflare supports by default. Cloudflare's free plan forwards to ports 80, 8080, 8880, 2052, 2082, 2086, 2095 (HTTP) and 443, 2053, 2083, 2087, 2096, 8443 (HTTPS).

Step 7: Access Mission Control through your clean URL

Once the connection is set up, open your browser and go to **https://mc.yourdomain.com** without any port number. No certificate warning, no port in the URL — just a clean, professional-looking URL with a trusted SSL certificate.

Extra security with Cloudflare Access: Once everything is set up through Cloudflare, you can optionally enable Cloudflare Access (free for up to 50 users). This lets you require login through Google, GitHub, or an email magic link before visitors even reach your basic auth. It's a second layer of protection and much more user-friendly than typing a password every time.

13. Troubleshooting

Here are the most common issues you might run into and how to fix them. These are the exact issues encountered during the live recording of this lesson.

Issue: Port 8443 is already in use

Symptom: Caddy fails to start with the error *bind: address already in use*.

Cause: Docker is using port 8443 for the Mission Control container.

Fix: Use port 8444 instead (this guide already uses 8444). If you followed along and accidentally used 8443, change it with:

```
sudo sed -i 's/:8443/:8444/g' /etc/caddy/Caddyfile
sudo systemctl restart caddy
```

Issue: SSL handshake fails (wrong version number)

Symptom: Browser or curl returns *SSL routines::wrong version number* or *tls1 alert internal error*.

Cause: Caddy's internal TLS generation failed due to sudoers permission issues with the caddy user.

Fix: Use the manual openssl certificate approach described in section 6 of this guide. That is what this guide already does by default.

Issue: Connection timeout in browser

Symptom: Browser shows *ERR_CONNECTION_TIMED_OUT* or *This site can't be reached*.

Cause: Your VPS firewall is blocking port 8444.

Fix: Follow section 3 of this guide to open port 8444 in your hosting provider's firewall panel, and remember to synchronize/apply the changes.

Issue: 502 Bad Gateway error

Symptom: After logging in, browser shows *HTTP ERROR 502 — This page isn't working*.

Cause: Mission Control is not running inside the Docker container on port 3007.

Fix: Follow section 9 of this guide to start Mission Control inside the container.

Issue: Caddy fails to start on port 80

Symptom: Caddy fails with *listen tcp :80: bind: address already in use*.

Cause: Caddy is trying to open port 80 for automatic HTTPS redirects, but something else (like Traefik) is already using it.

Fix: Add *auto_https off* to the global block at the top of your Caddyfile (this guide already does that).

Issue: PM2 command not found in container

Symptom: When running PM2 commands, you get *./node_modules/.bin/pm2: not found* or similar.

Cause: PM2 was not installed in this container, probably because the Task Board lesson was skipped or the container was recreated without PM2.

Fix: Install PM2 locally inside the container:

```
docker exec openclaw-ij7b-openclaw-1 sh -c \  
"cd /data/.openclaw/workspace/mission-control && \  
npm install pm2"
```

Then go back to section 9 step 3 and follow Outcome C to start Mission Control.

14. Useful commands reference

All commands below should be run in your SSH terminal, while connected to your VPS.

Check Caddy status

```
sudo systemctl status caddy --no-pager
```

View Caddy logs

```
sudo journalctl -xeu caddy.service --no-pager -n 50
```

Restart Caddy after config changes

```
sudo systemctl reload caddy
```

Check what's listening on a port

```
sudo ss -tlnp | grep 8444
```

View the current Caddyfile

```
sudo cat /etc/caddy/Caddyfile
```

Validate the Caddyfile

```
sudo caddy validate --config /etc/caddy/Caddyfile
```

List running Docker containers

```
docker ps
```

Check PM2 process list inside the container

```
docker exec openclaw-ij7b-openclaw-1 sh -c \  
"cd /data/.openclaw/workspace/mission-control && \  
./node_modules/.bin/pm2 list"
```

Restart Mission Control via PM2

```
docker exec openclaw-ij7b-openclaw-1 sh -c \  
"cd /data/.openclaw/workspace/mission-control && \  
./node_modules/.bin/pm2 restart mission-control"
```

View Mission Control logs via PM2

```
docker exec openclaw-ij7b-openclaw-1 sh -c \  
"cd /data/.openclaw/workspace/mission-control && \  
./node_modules/.bin/pm2 logs mission-control --lines 30 --nostream"
```

Congratulations!

You now have Mission Control running on a public URL, accessible from anywhere in the world, protected by basic auth and HTTPS. This marks the end of the Mission Control section of the course.

Appendix: Agent prompts used to design this setup

This appendix shows the exact prompts used with an AI agent to figure out the best deployment approach for Mission Control. If you want to do this yourself with your own agent, or if you want to understand why certain decisions were made (like using port 8443 instead of 80, or using a self-signed certificate instead of Let's Encrypt), read through the prompts below. They show the full decision-making process.

These prompts are copy-paste ready. You can send them to your OpenClaw agent (via Telegram or wherever you interact with it) to replicate the same decision process on your own VPS.

Prompt 1: Investigate what's using port 80

Before doing anything, check what's already running on the VPS so you don't break existing services:

```
The Docker deploy failed because port 80 is already in use on the VPS. Can you check what's running on port 80 and tell me what it is before we do anything? I want to know what service is using it so we can decide whether it's safe to stop or if we need a different approach.
```

Why this prompt matters: It tells the agent to investigate before acting. This prevented us from blindly killing a service that turned out to be the OpenClaw gateway infrastructure.

Prompt 2: Ask for a deployment approach that avoids port 80

Once you know port 80 is important, ask the agent to propose a solution that works around it:

```
Ok, let's not stop port 80. Can you set up Caddy to work without needing port 80 directly? Either use a DNS-01 challenge for the Let's Encrypt certificate, or route Mission Control through the existing OpenClaw gateway. What's the cleanest approach that doesn't conflict with the current container services?
```

Why this prompt matters: It gives the agent two options (DNS-01 or reverse proxy through OpenClaw) and asks it to recommend the cleanest one. This is better than picking one approach and forcing it.

Prompt 3: Request the specific Caddy setup

Once the agent recommends a path, give it specific requirements for the setup:

```
Set up Caddy as a reverse proxy for Mission Control using port 8443 since ports 80 and 443 are already taken by other services. Use a self-signed certificate that Caddy manages itself. Forward requests from port 8443 to localhost port 3007 where Mission Control is running. Add basic authentication with username admin and a strong random password that you generate. Show me the username and password clearly at the end of your response so I can save them. After making the changes, restart Caddy.
```

Why this prompt matters: It specifies port, certificate type, destination, auth method, and explicitly asks to display credentials at the end. Clear requirements lead to clear results.

Prompt 4: Approve the inspection step

If the agent asks for approval before making changes (which is good practice), approve with:

```
Approve Caddy inspection
```

Why this prompt matters: Agents often pause before making system-level changes. Respect that — it's a safety feature, not an obstacle.

Prompt 5: Request the one-shot install script

When the agent can't install things directly due to permissions, ask it for a script you can run yourself:

```
Give me the one-shot Caddy install script
```

Why this prompt matters: When an agent can't execute something itself, asking for a one-shot script is the fastest way to get a working solution. You run the script manually on your VPS, the script does everything in one go.

The one-shot install script (reference)

Here is the actual one-shot script that was generated by the agent. This script installs Caddy, generates a random password, creates the Caddyfile, and restarts the service all in one go. If you want to use it, paste it into your SSH terminal on the VPS. **Note:** this script uses port 8443 and Caddy's internal TLS, which had some issues in testing. The main guide uses port 8444 and manual openssl certificates which is more reliable — use the main guide for production, and use this script as reference for how the original setup was built.

```

set -euo pipefail

sudo apt update
sudo apt install -y caddy openssl

USERNAME="admin"
PASSWORD="$(openssl rand -base64 24 | tr -d '\n')"
HASH="$(caddy hash-password --plaintext "$PASSWORD")"

sudo mkdir -p /etc/caddy

sudo tee /etc/caddy/Caddyfile >/dev/null <<EOF
:8443 {
  tls internal

  basicauth {
    $USERNAME $HASH
  }

  reverse_proxy 127.0.0.1:3007
}
EOF

sudo caddy validate --config /etc/caddy/Caddyfile
sudo systemctl enable caddy
sudo systemctl restart caddy
sudo systemctl --no-pager --full status caddy || true

echo
echo "=====
echo "CADDY REVERSE PROXY READY"
echo "URL: https://$(hostname -I | awk '{print $1}'):8443"
echo "USERNAME: $USERNAME"
echo "PASSWORD: $PASSWORD"
echo "=====

```

Lessons from using the agent: Always ask the agent to investigate before acting. Give it clear requirements. When it can't execute something directly, ask for a script instead. And remember that the agent's solution might not work first time — the main guide in this PDF reflects the final working version after troubleshooting, which is why it differs slightly from the one-shot script above.